

IBM BOB 2.0 HACKATHON · LABLAB.AI · 25-27 SEP 2026

ForgeFlow

From a web-app idea to a verified live URL, with a self-healing repair loop that knows when to stop and hands the hard cases to IBM Bob.

TESTING

DEBUGGING

RELEASE & DEPLOYMENT

Faheemuddin Sayyed · Sanidhya Awasthi · Raghav Sonchhatra

Symbiosis Institute of Technology, Pune

01

Requirements

02

Design

03

Build

04



Test + Repair

05

Deploy

06

Validate

Generating a web app is fast. Proving that it works is still manual.

< 1 in 3

Infrastructure generated by two frontier models deployed successfully on the first attempt: 30.2% and 26.8% of tasks.

Zhang et al., arXiv:2506.05623 (2025)

1–3

Repair iterations deliver most of the gain. After that, a fix loop mostly spends tokens.

Kiecker et al.,
arXiv:2607.05197 (2026)
Arimbur, arXiv:2604.10508 (2026)

Less secure

Repeated rounds of AI self-refinement made code less secure in a controlled study. An open-ended fix loop adds risk.

Shukla et al., IEEE ISTAS (2025)

Today developers close that gap by hand: tests written after the code, CI logs pasted into a chat, fixes retried with no stopping rule, and a manual click-through of the live site.

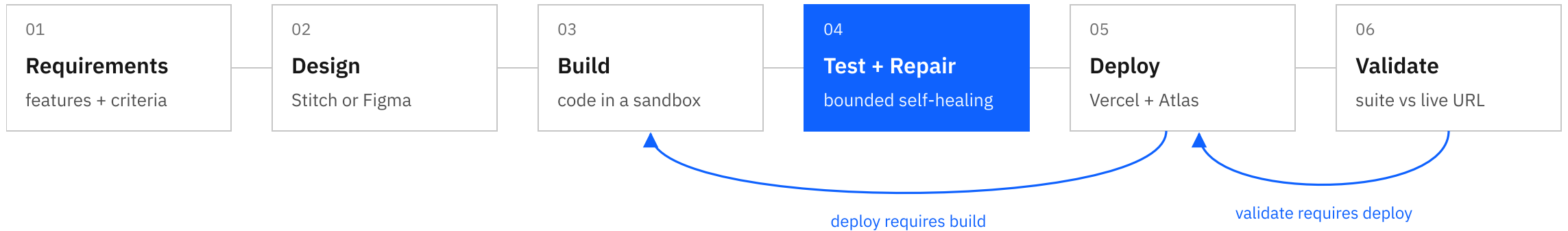
THE WORKFLOW WE IMPROVE

Verify, repair, release: the part of the job that AI app builders leave to you

	TODAY
TESTS	Written by hand after the code, if at all
DEBUG	Paste failing output into a chat and guess which files matter
RETRY	Ask again until it works or the budget runs out
RELEASE	Set up hosting, environment variables and a database by hand
VALIDATE	Click through the live site and hope

WITH FORGEFLOW
Generated from your acceptance criteria, each test tagged with its criterion ID
The failure analyzer selects the failing tests, the sources they exercise and the diff since the last green run
Four guards stop the loop, and a stuck summary goes to a human
The infra analyzer picks targets: SPA and API to Vercel, data to MongoDB Atlas
The same generated suite re-runs against the live URL

Six stages. Enter, skip or revisit any of them.



Each stage keeps its own state

Status and versioned artifacts per stage. Nothing is overwritten.

Stale, never deleted

Refining requirements marks downstream work stale, and you decide what to redo.

Only two hard rules

Start from screenshots, a structured spec, or go straight to Build. The blue arcs are the only prerequisites.

A repair loop that knows when to stop

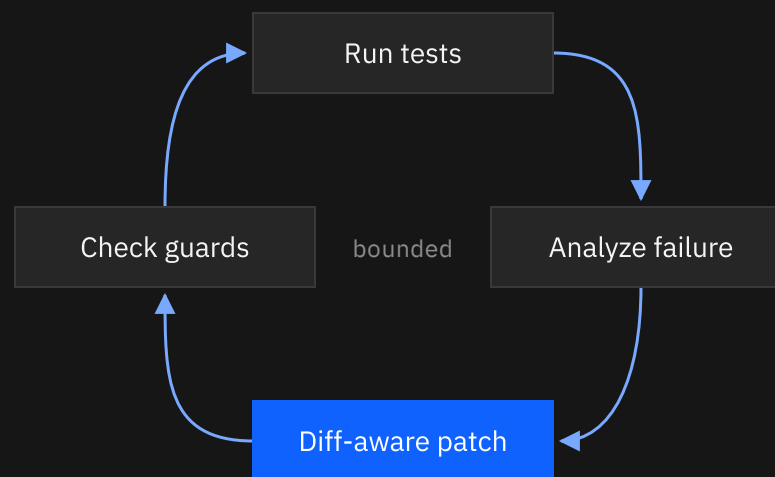
WHAT THE REPAIR AGENT SEES

The failing tests

Only the sources those tests exercise

The git diff since the last green run

The acceptance criterion each test checks



STOPS AT THE FIRST OF

Regression guard

a patch breaks a passing test

No-progress guard

the failing count stops shrinking

Iteration cap

Budget cap

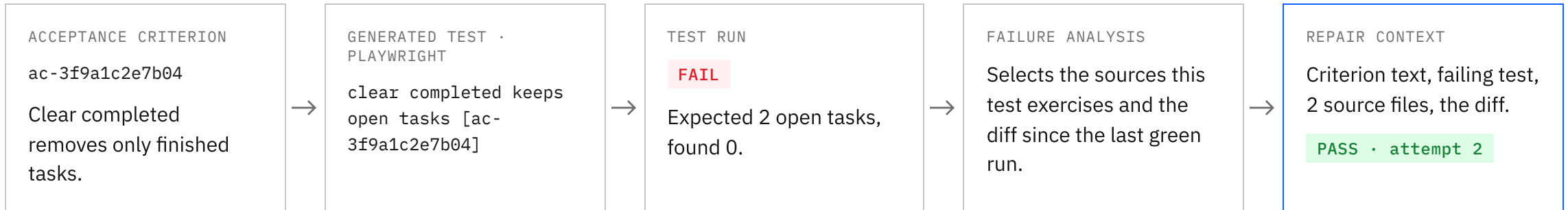
tokens and cost, per project

Then it escalates to a human.

ESCALATION SUMMARY · THE CONTROLLER'S REAL FORMAT, EXAMPLE VALUES

I stopped after 3 attempt(s) because the failing tests stopped shrinking. 1 test(s) are still failing: clear completed keeps open tasks [ac-3f9a1c2e7b04]. I patched: backend/src/routes/todos.ts, frontend/src/App.tsx. Failing count per attempt: 4 → 1 → 1. Tell me what to change and I'll rebuild from your guidance.

One criterion ID links the requirement, the test, the failure and the fix



The ID follows the app into production

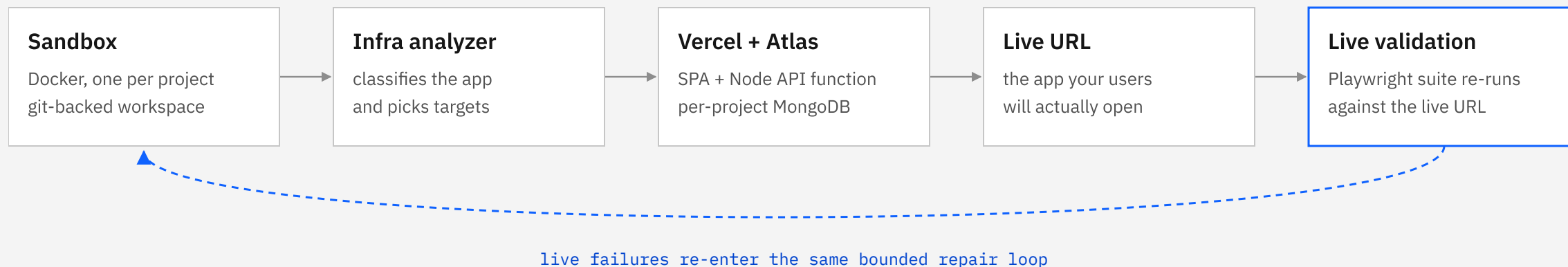
Live-validation results carry the same criterion ID, so a failure on the deployed URL points back to the requirement it breaks.

Example values from the Todo demo.

How the link is made

Criteria are minted as `ac-<12 hex>`. Tests carry the ID in their title, and the result parser reads it back from each run.

Deployed, then tested again where users will see it



No git host in the path

The API ships as a Vercel function built from an inline source upload.

Secrets stay server-side

Provider tokens are encrypted at rest, decrypted only at call time, and never returned to the browser.

Host choice by setting

Render stays available as the backend host behind one configuration value.

ForgeFlow automates the bounded part. IBM Bob takes over where it stops.

IBM Bob as our development partner

ForgeFlow is a real, multi-service codebase of about 800 files: a FastAPI control plane, a React PWA, Docker sandboxes and deploy adapters. During the hackathon we worked on it in IBM Bob IDE, using Bob 2.0's full-repository context to onboard, review and extend it. The repo ships `AGENTS.md` and `.bob/rules/` so Bob loads our conventions in every session.

Continue in IBM Bob in progress

When the repair loop escalates, ForgeFlow exports the workspace ready for Bob:

<code>AGENTS.md</code>	stack, commands and conventions
<code>.bob/rules/forgeflow-stack.md</code>	the fixed generated-app stack
<code>BOB_HANDOFF.md</code>	failing tests, criteria, patches tried, failing-count trail

Open the folder in IBM Bob IDE, start in Plan mode, and keep going with everything ForgeFlow learned.

What we did in IBM Bob IDE this weekend

#	TASK	IBM BOB 2.0 FEATURE	RESULT IN THE REPO	BOBCOINS
01	Onboard Bob to an 800-file, multi-service repo	/init, Ask mode, parallel explore subagents	<code>AGENTS.md</code> and <code>.bob/</code> mode context	[]
02	Review the repair loop, sandbox and deploy code	/review and the Bob Findings panel	Fix commits for confirmed findings	[]
03	Plan the IBM Bob handoff from our own specs	Plan mode reading the implementation plan and phase specs	<code>plans/phase-66-bob-handoff.md</code>	[]
04	Build the handoff end to end	Agent mode with parallel tool calls	Export endpoint and the Continue in IBM Bob button	[]
05	Test the handoff	Agent mode with a general subagent	pytest and Vitest cases	[]

Evidence: every session's consumption-summary screenshot and exported task history, for each team member, in `bob_sessions/` .

For people who ship web apps faster than they can verify them

Student and indie builders

Go from screenshots or a short spec to a tested, deployed full-stack app without building a test harness or a deploy pipeline.

Agencies and small product teams

Every client prototype ships with a test suite traced to its acceptance criteria and a live-validation report.

Teams adopting AI coding tools

Generated code is verified with a stopping rule, a cost record per project, and a human decision at the end.

Cost discipline is built in

Models are routed by role. A small model classifies and summarizes; the code model runs only for code, tests and repair. Token use and cost are recorded per project against a budget cap. When money is tight, ForgeFlow skips planning before it skips verification.

Working today, with the evidence in the repository

2,148

automated tests passing across the control plane, PWA, sandbox and deploy code (19 Sep 2026)

65

build phases shipped, each with written acceptance criteria and passing tests

6

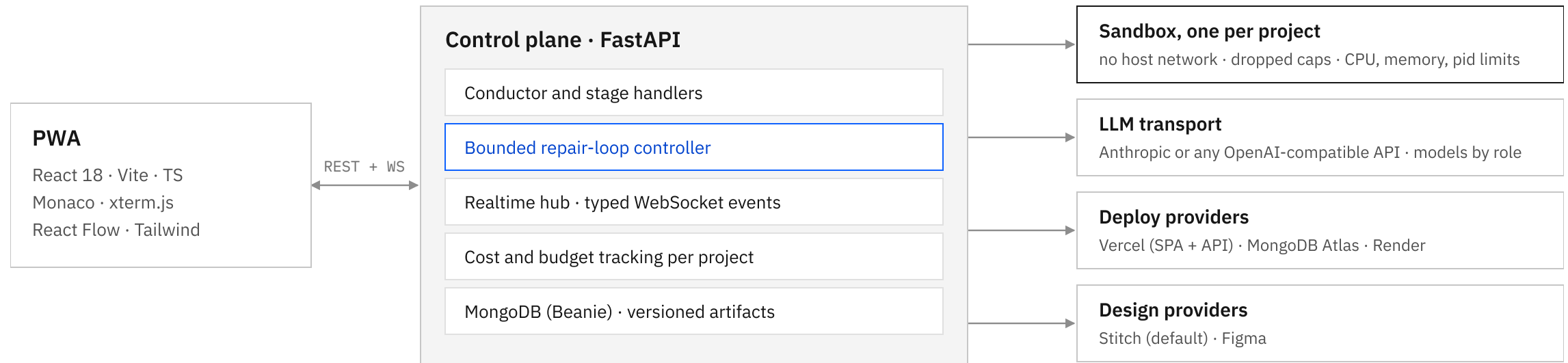
CI jobs gate every change: backend, frontend, sandbox, app skeleton, security and config

10

benchmark apps. Each run records first-pass and post-repair pass rate, iterations, regressions and cost.

Every phase clears the same gate: ruff, black and mypy on the backend; ESLint, Prettier and Vitest on the PWA; pytest against a disposable MongoDB.

Untrusted code only ever runs inside a sandbox



Every generated app uses one stack: React + Vite + TS · Express + TS · MongoDB (Mongoose) · Zod · Vitest · Jest + supertest · Playwright.

DEMO

Screenshots in, a verified live URL out

- 01 Describe a todo app, or drop in screenshots
- 02 Refine the design with one sentence
- 03 Build full-stack code in the sandbox, with live preview
- 04 Generate tests from the acceptance criteria
- 05 Watch the repair loop fix Clear completed
- 06 Deploy the SPA and API to Vercel, data to MongoDB Atlas
- 07 Re-run the suite against the live URL
- 08 Next: [escalate a hard case and continue it in IBM Bob](#)

LIVE APP

forgeflow-ecru.vercel.app

VIDEO DEMO

youtu.be/cjVYj87INMo

CODE

github.com/Islinger19/ForgeFlow-IBM-BOB

IBM BOB EVIDENCE

[bob_sessions/](#) in the repository

TEAM

Faheemuddin Sayyed, Sanidhya Awasthi, Raghav Sonchhatra